7. HAFTA:Yazılım Geliştirme Süreç Modelleri

Şelale Modeli, Çevik Yöntemler, Spiral ve V-Modeli



Şelale(Çağlayan) Modeli: Tanım ve Temel Felsefe

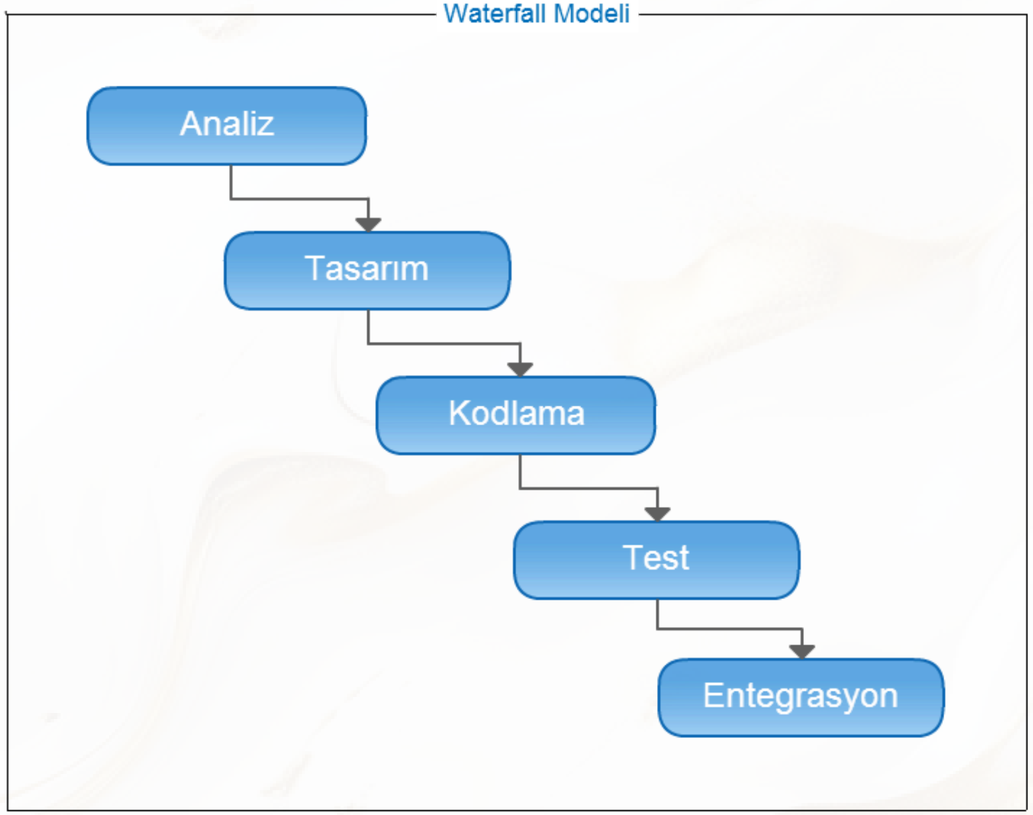
Şelale Modeli (Waterfall Model), yazılım geliştirme sürecini yönetmek için kullanılan en eski ve en temel metodolojilerden biridir. Adını, bir şelalenin basamaklarından aşağı doğru akan suyun geri dönememesinden alan bu model, proje aşamalarının katı bir sıralama ile birbirini takip ettiği doğrusal (linear) ve ardışık (sequential) bir yapıya dayanır.



Modelin temel felsefesi, bir sonraki aşamaya geçmeden önce mevcut aşamanın eksiksiz bir şekilde tamamlanması ve onaylanmasıdır. Bu yaklaşım, projenin en başında tüm gereksinimlerin detaylı bir şekilde tanımlanmasını, belgelenmesini ve bu plana proje sonuna kadar sadık kalınmasını gerektirir.

Şelale(Çağlayan) Modeli: Tanım ve Temel Felsefe

Birçok modern metodoloji, Şelale modelinin kısıtlamalarına bir tepki olarak veya onun temel prensipleri üzerine bir geliştirme olarak ortaya çıkmıştır. Bu nedenle, Şelale modelini anlamak, günümüz proje yönetimi paradigmalarının evrimini ve temel mantığını kavramak için kritik bir öneme sahiptir.



Şelale Modeli, Yazılım Geliştirme Yaşam Döngüsü'nün (Software Development Life Cycle - SDLC) klasik bir uygulamasıdır. SDLC'nin temel aşamaları olan gereksinim analizi, sistem tasarımı, uygulama (kodlama), test, dağıtım ve bakım gibi adımları, katı bir hiyerarşi ve sıralama ile uygular. Her aşama, bir öncekinin çıktısını girdi olarak kullanır ve kendi çıktısını bir sonraki aşamaya devreder. Bu yukarıdan aşağıya doğru kesintisiz akış, modele adını veren "şelale" metaforunu oluşturur.

Şelale Modelinin Avantajları

Yapısal Netlik ve Kontrol

Her aşamanın açıkça tanımlanmış başlangıç noktaları, bitiş noktaları ve somut teslimatları bulunur. Proje yöneticileri için ilerlemeyi izlemeyi, kaynakları yönetmeyi ve sürecin kontrolünü sağlamayı son derece kolaylaştırır.

Kapsamlı Dokümantasyon

Her aşamada detaylı dokümantasyon üretilmesi zorunludur. Bu belgeler birincil iletişim aracı olarak görev yapar ve projenin ilerleyen dönemlerindeki bakım ve güncelleme süreçleri için paha biçilmez bir kaynak oluşturur.

Öngörülebilirlik

Projenin en başında tüm gereksinimlerin net bir şekilde tanımlanıp "dondurulması", bütçe ve zaman yönetimi açısından yüksek doğruluk payına sahip tahminler yapılmasını sağlar.

Risk Yönetimi

Her aşamanın tamamlanıp doğrulanması, teorik olarak, bir sonraki aşamaya geçmeden önce risklerin tespit edilip çözülmesine olanak tanır. Sorumluluk takibi daha kolay hale gelir.

Yapısal Netlik ve Kontrol

Bu katı ve disiplinli çerçeve, özellikle kendi başına inisiyatif alma veya karmaşık süreçleri yönetme konusunda tecrübesi olmayan proje elemanları veya performansı dalgalı ekipler için ideal bir çalışma ortamı sunar. Herkesin rolü ve görevleri net bir şekilde tanımlandığı için, proje ekibine her şeyin kontrol altında olduğu ve projenin iyi yönetildiği hissini verir.

> Bu kontrol mekanizması, özellikle büyük kurumsal yapılar ve devlet kurumları için büyük bir çekiciliğe sahiptir. Bu tür organizasyonlar, genellikle süreç kontrolünü, öngörülebilirliği ve denetlenebilirliği, esneklik ve adaptasyona tercih ederler.



Şelale Modelinin Dezavantajları:

Esneklik Eksikliği

Modelin katı ve doğrusal doğası, proje sürecinde ortaya çıkabilecek değişikliklere karşı son derece dirençlidir. Bir aşama tamamlanıp onaylandıktan sonra geriye dönüp değişiklik yapmayı neredeyse imkansız hale getirir.

Gecikmiş Geri Bildirim

Müşteri ve son kullanıcı geri bildirim döngüsünün tehlikeli derecede geç işlemesi. Müşteri, projenin en başında gereksinimleri belirler ve genellikle sürecin sonundaki test veya teslimat aşamasına kadar sürece aktif olarak dahil olmaz.

Yüksek Risk Konsantrasyonu

Tüm test, entegrasyon ve doğrulama faaliyetleri projenin son aşamalarına yığılır. Erken analiz veya tasarım aşamalarında yapılan kritik bir hata, ancak kodlamanın tamamı bittikten sonra fark edilir.

İnovasyon Engeli

Katı bir şekilde orijinal plana odaklanma, proje sürecinde ortaya çıkabilecek yenilikçi fikirleri ve yaratıcı çözümleri engeller. Değişiklikler birer sorun olarak görülür ve caydırılır.

Şelale Modeli: Kullanım Senaryoları

Sabit ve Net Gereksinimler

Proje gereksinimlerinin başlangıçta tam, net, anlaşılır ve proje süresince değişmeyeceğinden emin olunan durumlar. Yasal düzenlemelere uyum, askeri projeler veya bilimsel hesaplama yazılımları.

Geleneksel Mühendislik

Köprü, bina veya üretim hattı inşaatı gibi fiziksel ürünlerin geliştirildiği projeler. Aşamalar doğası gereği ardışıktır ve tasarım sonrası değişiklikler son derece maliyetlidir.

Legacy Sistemler

Mevcut, kararlı ve iyi belgelenmiş eski sistemlerin bakımı veya küçük, iyi tanımlanmış güncellemeler. Amaç mevcut işlevselliği korumak ve sistem kararlılığını bozmamaktır.

Küçük ve Basit Projeler

Gereksinimleri iyi tanımlanmış, kapsamı sınırlı ve teknik karmaşıklığı düşük olan küçük ve orta ölçekli projeler. Basit kurumsal web siteleri veya temel otomasyon uygulamaları.

Çevik Metodoloji Nedir?

Çeviklik, günlük işlerdeki değişikliklere ve zorluklara hızlı, esnek ve proaktif bir şekilde uyum sağlamaya odaklanan belirli bir zihniyeti ifade eder.

Çevik proje yönetimi, bu zihniyetten kaynaklanan ve esnekliğe, sürekli öğrenmeye ve uyum sağlamaya dayanan çalışma biçimini tanımlar. Değişen gereksinimlere uyum sağlayabilmek için iş süreçleri her zaman yeterince esnek tutulmalıdır. İnsanlar ve etkileşimler, araçlar ve süreçlerden daha önemlidir. Müşteri memnuniyeti nihai hedeftir.



Çevik Manifestosu

Çevik Manifestosu (Agile Manifesto), 2001 yılında 17 yazılım uzmanı tarafından yayımlanmış bir bildiridir ve modern yazılım geliştirme yaklaşımlarının temel felsefesini tanımlar. Agile yaklaşımının resmî ideolojik temeli kabul edilir. Bu manifesto, dört temel değeri ve on iki prensibi içerir.



Çevikliğin 4 Temel İlkesi

Bireyler ve Etkileşimler

Çevik ekipler, bağımsız çalışmaktan ve işleri "kitaba göre" yapmaktan ziyade ekip iş birliğine ve takım çalışmasına değer verir.

Çalışan Yazılım

Çevik ekiplerin geliştirdiği yazılımlar işe yaramalıdır. Dokümantasyon gibi ek çalışmalar, iyi bir yazılım geliştirmek kadar önemli değildir.

Müşteri İş Birliği

Çevik ekipler, müşterilerin yazılımın nereye gitmesi gerektiğine karar vermelerine olanak tanır. Müşteri iş birliği, sözleşme müzakeresinin ince ayrıntılarından daha önemlidir.

Değişime Yanıt

Bu çerçeve, ekiplerin tüm bir projeyi rayından çıkarmadan stratejilerini ve iş akışlarını hızla değiştirmelerine olanak tanır.

12 Çevik Prensip Nelerdir?

Çevik modelin dört değeri bir evin ağırlık taşıyan sütunlarıysa, 12 Çevik ilke de o evin içinde inşa edebileceğiniz odalardır. Bu ilkeler, yazılım geliştirme sürecinizin ihtiyaçlarına kolayca uyarlanabilir.

1. Erken ve Sürekli Değer Teslimi:

En önemli öncelik, değerli yazılımın erken ve devamlı teslimiyle müşteriyi memnun etmektir. Bu, projenin sonunda büyük bir teslimat yapmak yerine, çalışan ürün parçalarını sık aralıklarla sunmak anlamına gelir.

2.Değişimi Kucaklamak:

Değişen gereksinimler, geliştirme sürecinin son aşamalarında bile kabul edilmelidir. Çevik süreçler, değişimi müşterinin rekabet avantajı için bir araç olarak kullanır.

3.Sık Teslimat

1. ilkeye benzer şekilde, müşterilerinize veya paydaşlarınıza sürekli değer sunmak, onların müşteri kaybetme olasılığını azaltır.

4.İş Birliği

İşlevler arası ekipler ve iş birliği, Çevikliğin temel değerlerinden biridir. Amaç, insanların bireysel projelerinden çıkıp daha sık iş birliği yapmalarını sağlamaktır .

12 Çevik Prensip Nelerdir?

5.Motivasyon

Projelerin temelinde motive olmuş bireyler yer almalıdır. Onlara ihtiyaçları olan ortam ve destek sağlanmalı ve işi başaracakları konusunda güven duyulmalıdır.

6.Yüz Yüze İletişim

Bir yazılım takımında bilgi alışverişinin en verimli ve etkin yöntemi yüz yüze iletişimdir.

7.Çalışan Yazılım

Çalışan yazılım, ilerlemenin temel ölçütüdür. Yazılım geliştirme projelerinin nihai hedefi çalışan bir üründür ve Agile çerçevesi, her şeyden önce işlevsel yazılıma öncelik vererek bunu destekler.

8.Sürdürülebilir bir çalışma temposu

Çevik proje yönetiminin bazı yönleri hızlı tempolu olabilir, ancak ekip üyelerinin tükenmesine neden olacak kadar hızlı olmamalıdır. Amaç, geliştirme süreci boyunca sürdürülebilirliği sağlamaktır.

12 Çevik Prensip Nelerdir?

9.Teknik Mükemmeliyet:

Teknik mükemmeliyet ve iyi tasarım konusundaki sürekli özen, çevikliği artırır.

10.Basitlik esastır.

Bazen en basit çözüm en iyi çözümdür. Çevik geliştirme, işleri gereğinden fazla karmaşıklaştırmamayı ve karmaşık sorunlara basit çözümler bulmayı hedefler.

11.Kendi kendini organize eden ekipler

En iyi mimariler, gereksinimler ve tasarımlar, kendi kendini organize eden takımlardan ortaya çıkar.

12.Sürekli İyileştirme (Kaizen):

Takım, düzenli aralıklarla nasıl daha etkili ve verimli olabileceğinin üzerinde düşünür ve davranışlarını buna göre ayarlar ve düzenler.

Çevik Geliştirme Metodolojisinin Faydaları

Uyarlanabilirlik

Çevik yöntemler uyarlanabilirdir. Yazılım geliştirmede Çevik süreçleri kullanmanın temel faydalarından biri, bir projenin akışını aksatmadan stratejileri hızla değiştirebilme olanağı sağlamasıdır.
Geleneksel şelale yöntemindeki aşamalar birbirinin içine geçtiğinden, stratejileri değiştirmek zorlu olabilir ve proje yol haritasının geri kalanını bozabilir.

İşbirlikçi Ekip Çalışması

Çeviklik, işbirlikçi ekip çalışmasını teşvik eder. Çevik ilkelerden biri, ekibinizle iletişim kurmanın en etkili yolunun yüz yüze görüşmek olduğunu belirtir.

Bunu, ekipleri proje silolarını kırmaya teşvik eden ilkeyle birleştirdiğinizde, iş birliğine dayalı bir ekip çalışması için bir formül elde edersiniz.

Müşteri Odaklılık

Çevik yöntemler müşteri ihtiyaçlarına odaklanır. Yazılım geliştirmenin benzersiz yönlerinden biri, ekiplerin müşteri ihtiyaçlarına diğer sektörlere kıyasla çok daha yakından odaklanabilmesidir.

Müşteri memnuniyeti yazılım geliştirmenin temel itici gücü olduğundan, Çevik sürece neden dahil edildiğini anlamak kolaydır.

Çevik Metodolojilerin Türleri

Çevik çerçeve, birçok farklı varyasyonu kapsayan bir çatıdır. İşte en yaygın Çevik metodolojilerden birkaçı.

Kanban



Kanban, Agile'a görsel bir yaklaşımdır. Ekipler, belirli görevlerin geliştirme sürecindeki yerini temsil etmek için çevrimiçi Kanban panosu araçlarını kullanır. Görevler bir panodaki kartlarla, aşamalar ise sütunlarla temsil edilir.

Scrum



Scrum, küçük ekipler için yaygın bir Çevik metodolojidir ve sprint'leri de içerir. Ekip, günlük işleri yürüten diğer ekipler için tüm engelleri ortadan kaldırmak olan bir Scrum yöneticisi tarafından yönetilir.

Aşırı Programlama (XP)



Genellikle yazılım geliştirmede kullanılan Extreme Programming (XP), ekibinizin daha etkili bir şekilde birlikte çalışmasını sağlayacak değerleri özetleyen çevik bir çerçevedir. XP'nin beş değeri: İletişim, Basitlik, Geri bildirim, Cesaret ve Saygı.

Çevik Metodolojilerin Türleri

Uyarlanabilir Proje Çerçevesi (APF)

Uyarlanabilir Proje Çerçevesi, aynı zamanda Uyarlanabilir Proje Yönetimi (APM) olarak da bilinir ve bir proje sırasında bilinmeyen faktörlerin herhangi bir anda ortaya çıkabileceği fikrinden doğmuştur. APF, bir projenin ihtiyaç duyduğu kaynaklara değil, sahip olduğu kaynaklara odaklanır.

Aşırı Proje Yönetimi (XPM)

Bu proje yönetimi türü, genellikle yüksek belirsizlik içeren çok karmaşık projelerde kullanılır. Bu yaklaşım, istenen sonuca ulaşana kadar süreçlerin sürekli olarak uyarlanmasını içerir. XPM çok fazla esneklik gerektirir.

Uyarlanabilir Yazılım Geliştirme (ASD)

Bu Çevik metodoloji, ekiplerin değişen gereksinimlere hızla uyum sağlamasını sağlar. Bu sürecin temel odağı sürekli adaptasyondur. Bu proje türünün aşamaları -tahmin etme, iş birliği yapma ve öğrenme- proje ilerledikçe sürekli öğrenmeye olanak tanır.



DSDM ve FDD Metodolojileri

Dinamik Sistem Geliştirme Yöntemi (DSDM)

Dinamik Sistem Geliştirme Yöntemi, tüm proje yaşam döngüsüne odaklanan bir Çevik yöntemdir. Bu nedenle, DSDM diğer Çevik yöntemlerden farklı olarak daha titiz bir yapıya ve temele sahiptir.

DSDM'nin dört ana aşaması:

- Fizibilite ve iş çalışması
- İşlevsel mod veya prototip yinelemesi
- Tasarım ve yapım yinelemesi
- Uygulama

Özellik Odaklı Geliştirme (FDD)

Özellik Odaklı Geliştirme, farklı Çevik en iyi uygulamalarını harmanlar. Yinelemeli bir proje yönetimi yöntemi olsa da, bu model daha çok ekibin geliştirmeye çalıştığı yazılımın özelliklerine odaklanır.

Özellik odaklı geliştirme, büyük ölçüde müşteri girdisine dayanır, çünkü ekibin önceliklendirdiği özellikler, müşterilerin ihtiyaç duyduğu özelliklerdir.

Bu model aynı zamanda ekiplerin projeleri sık sık güncellemelerine olanak tanır.

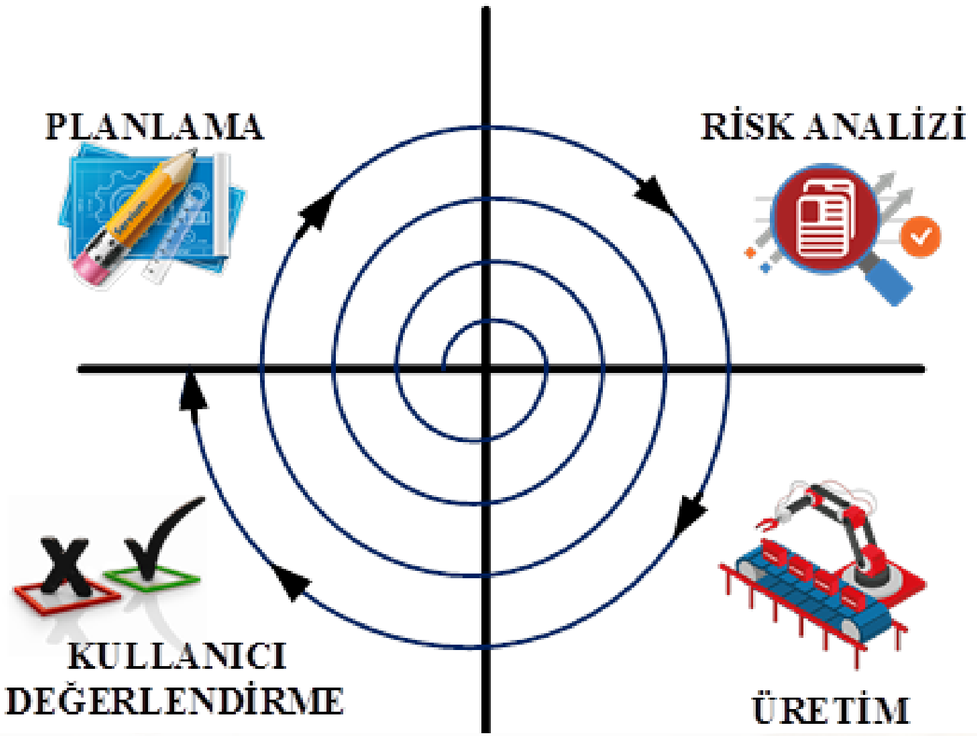


UYGULAMA

Çevik metodolojiler ile Şelale modelini tablo halinde karşılaştırın. Kendi bir yazılım projenizde hangisini tercih ederdiniz ve nedenini açıklayın.

Spiral Model

Spiral Model, özellikle büyük, maliyeti yüksek, riskli yazılım projelerinde kullanılan risk odaklı ve iteratif bir yazılım geliştirme modelidir. Bu modelde geliştirme süreci “spiral” şeklinde ilerler; yani her döngüde proje biraz daha geliştirilir, her turda mutlaka risk analizi yapılır.

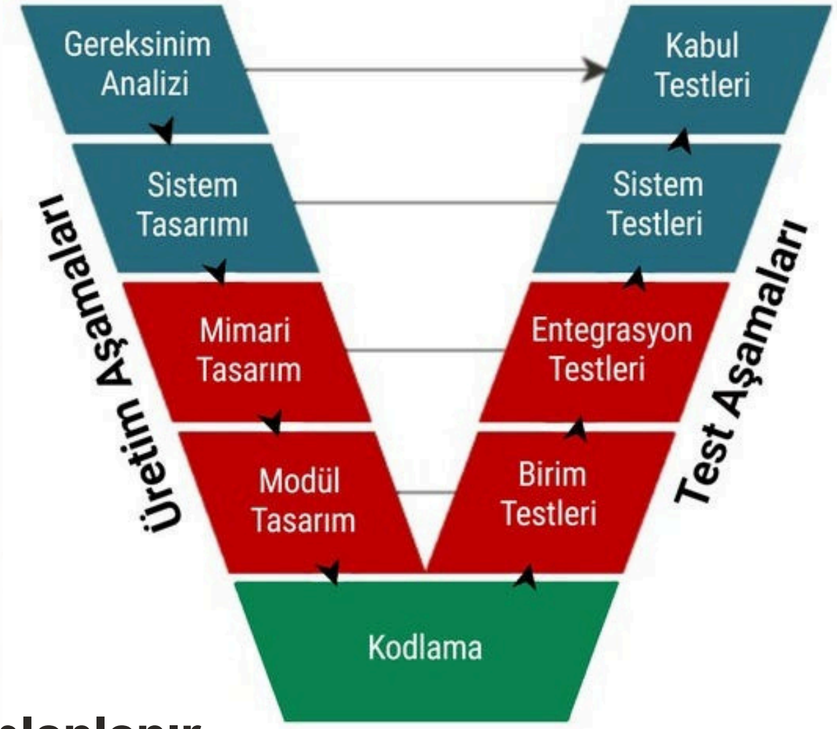


Temel Özellikleri

- **Her aşaması döngüseldir (iteration).**
- **Her iterasyonda risk analizi yapılması zorunludur.**
- **Önce küçük, düşük riskli parçalar geliştirilir; zamanla daha karmaşığa geçilir.**
- **Agile gibi esnektir, ama çok daha mühendislik ağırlıklıdır.**
- **Proje müşteriye her döngü sonunda prototip olarak sunulabilir.**

V-Modeli (Verification & Validation Model)

V-Model, Şelale Modelinin daha gelişmiş ve “test odaklı” versiyonudur. Geliştirme süreci V harfi şeklinde ilerler; sol tarafta geliştirme adımları, sağ tarafta onların birebir karşılığı olan test adımları yer alır. Yani her geliştirme aşamasının karşısında önceden planlanmış bir test aşaması vardır.



Temel Mantık

- **Şelale gibi sıradan bir akış değil , geliştirme ile birlikte test de planlanır.**
- **Daha proje başında hangi aşamada nasıl test yapılacağı belli olur.**
- **Özellikle hataları proje sonunda değil, başta yakalamayı hedefler.**